

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
Unidade Leopoldina
Engenharia de Computação

Trabalho 02 - Profiling

TEFEC – Computação Paralela

Caio Rangel Gomes

Guilherme Henrique de Paula Pereira França

Davi Marques de Oliveira

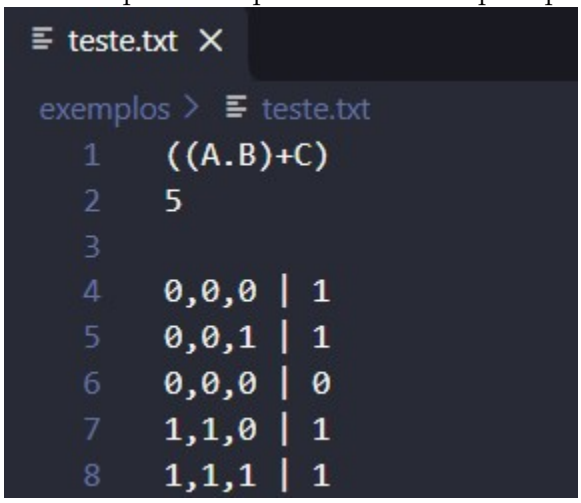
José Eduardo Henriques da Silva

1 Descrição do projeto e da versão sequencial

O projeto tem por objetivo validar entradas de tabela verdade inseridas pelo usuário. Ele recebe uma expressão lógica e sua tabela verdade como entrada (por meio de um arquivo .txt) e apresenta na saída quantas daquelas entradas estão corretas e quantas estão erradas, juntamente com uma porcentagem de acerto. O projeto aceita tanto tabelas verdades incompletas quanto completas, e apresenta um erro na saída caso a entrada não esteja no formato desejado.
O funcionamento do algoritmo se dá em 4 etapas. A etapa 1 consiste na leitura do arquivo txt, armazenando a expressão em uma string, o numero de casos em um int, as variáveis em um map e vector e o numero de variáveis em um int.
A etapa 2 consiste em construir a arvore binária. Essa etapa monta um parser com os dados da expressão e coloca eles na arvore.
Na etapa 3, cada caso de teste é inserido na arvore e compara o resultado com o fornecido no arquivo txt para aquele caso. Caso esteja certo, acrescenta 1 na variavel de acertos, caso esteja errado acrescenta 1 na variavel de erros
Na etapa 4 consiste em apresentar os resultados na tela, com a porcentagem de acertos

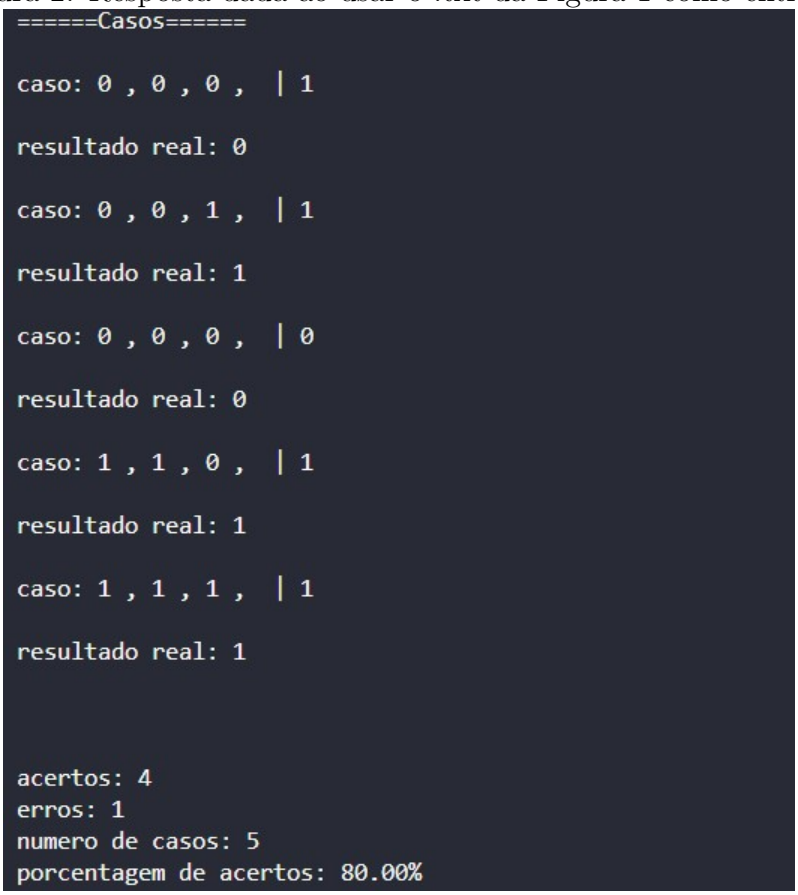
Abaixo é mostrado um exemplo de arquivo .txt de entrada para o programa, que representa uma expressão lógica de 3 variáveis e os casos de teste da tabela verdade que o usuário deseja que sejam validados:

Figura 1: Exemplo de arquivo .txt aceito pelo programa



Abaixo segue a resposta do programa para a entrada anterior:

Figura 2: Resposta dada ao usar o .txt da Figura 1 como entrada



Apesar de funcional, o programa apresenta problemas em relação à sua escalabilidade, tendo altos tempos de execução que escalam com o número de casos de teste para a tabela verdade.

O maior teste feito no programa foi com uma tabela verdade completa para uma expressão de 15 variáveis, apresentando 32768 casos de teste. Esse teste levou 8,592 segundos para ser executado, um aumento considerável quando comparado aos 0,007 segundos que o .txt da Figura 1 levou para ser executado.

2 Ambiente de medição

Todos os testes foram realizados em um computador com as seguintes especificações:

- Processador:
  - Modelo: 13th Gen Intel(R) Core(TM) i7-13620H (2.40 GHz)
  - Núcleos lógicos: 16
  - Núcleos físicos: 10
- Memória RAM:

  - Modelo: 8 Gb 5200 MT/s
- Sistema Operacional:
  - Edição: Linux Ubuntu
  - Versão: 26.04.1
- Compilador e versão:
  - Compilador: ubuntu1
  - Versão: 15.2.0
- Flags de compilação: -O2
- Data e horário das medições: 15:00 da tarde, 08/11/2026

3 Metodologia

O objetivo geral deste trabalho é analisar o desempenho do código por meio de ferramentas de profiling para encontrar os chamados hotspots, pontos que necessitam de maior processamento e tempo para serem executados. Assim, as ferramentas escolhidas foram o Callgrind e o cachegrind, ambas ferramentas presentes no sistema operacional Linux. Essas duas ferramentas foram escolhidas por apresentarem juntas todos os dados necessários para uma boa análise do algoritmo proposto.
O Callgrind analisa e cria um arquivo contendo quais linhas e funções do código são mais chamadas, e por consequência exigem mais processamento. Esse arquivo pode ser usado posteriormente para a criação de gráficos, apresentando os dados de forma visual e de fácil entendimento.
Já o cachegrind é usado para analisar as instruções quem compõe o código, e foi usado nas análises principalmente para forneces os dados de branch mispredictions e cache misses.
Para gerar a estatística associada a cada caso de teste, foi proposto executar o programa 10 vezes com cada caso, para que dados como a média, mediana e desvio padrão sejam levados em consideração na análise. O tempo de execução de cada etapa do codigo também foi gerado para cada teste, usando a biblioteca chrono.
Por fim, foram usados 4 casos de testes, visando usar as duas funções disponíveis no programa, com altas e baixas cargas de tabela verdade para teste. Os testes realizados foram os seguintes:

- teste normal:
  - teste com 20 variáveis e 10 mil entradas
  - teste com 20 variáveis e 50 mil entradas
- teste tabela completa (mega brain):
  - teste com 12 variáveis e 4.096 entradas
  - teste com 14 variáveis e 16.384 entradas

4 Resultados do profiling

Esta seção trará os resultados do profiling em tabelas, divididos para cada caso de teste. Os *flame graphs* poderão ser encontrados no Anexo I, tendo em vista que a grande largura das imagens impede sua clara visualização no documento.

4.1 Teste com 20 variáveis e 10 mil entradas

O primeiro teste realizado foi o teste com 10 mil entradas. O teste foi executado 10 vezes e os resultados encontrados podem ser encontrados na tabela abaixo:

Tabela 1: Medições de profiling: normal 10k

Execução / estatística	Etapa 1 (s)	Etapa 2 (s)	Etapa 3 (s)	Etapa 4 (s)	Tempo total (s)
1	0,014	0,01	18,044	0,011	18,079
2	0,012	0,008	18,077	0,011	18,108
3	0,012	0,008	17,849	0,011	17,88
4	0,015	0,015	18,407	0,011	18,448
5	0,013	0,016	18,217	0,012	18,258
6	0,011	0,013	17,917	0,012	17,953
7	0,012	0,012	18,102	0,011	18,137
8	0,012	0,009	17,74	0,011	17,772
9	0,013	0,012	17,822	0,012	17,859
10	0,013	0,009	17,708	0,012	17,742
média	0,0127	0,0112	17,9883	0,0114	18,0236
mediana	0,0125	0,011	17,9805	0,011	18,016
Desvio padrão (populacional)	0,0011	0,002712931993	0,2106836728	0,0004898979486	0,2129343561
mínimo	0,011	0,008	17,708	0,011	17,742
máximo	0,015	0,016	18,407	0,012	18,448
primeiro quartil	0,012	0,009	17,82875	0,011	17,86425
terceiro quartil	0,013	0,01275	18,09575	0,012	18,12975
cache misses	112914 (86745 rd + 26169 wr)				
mispredict	9787289 (9384801 cond + 402,488 ind)				

4.2 Teste com 20 variáveis e 50 mil entradas

O segundo teste realizado foi o teste com 50 mil entradas. O teste foi executado 10 vezes e os resultados encontrados podem ser encontrados na tabela abaixo:

Tabela 2: Medições de profiling: normal 50k

Execução / estatística	Etapa 1 (s)	Etapa 2 (s)	Etapa 3 (s)	Etapa 4 (s)	Tempo total (s)
1	0,011	0,008	89,971	0,013	90,004
2	0,0112	0,009	88,95	0,012	88,983
3	0,016	0,01	91,123	0,012	91,16
4	0,012	0,008	91,763	0,014	91,797
5	0,014	0,009	90,159	0,011	90,193
6	0,014	0,009	88,917	0,011	88,951
7	0,015	0,012	91,451	0,013	91,49
8	0,012	0,008	92,278	0,013	92,311
9	0,013	0,009	89,799	0,016	89,836
10	0,013	0,01	91,168	0,012	91,203
média	0,01312	0,0092	90,5579	0,0127	90,5928
mediana	0,013	0,009	90,641	0,0125	90,6765
Desvio padrão (populacional)	0,001552288633	0,001166190379	1,107140231	0,001417744688	1,107590701
mínimo	0,011	0,008	88,917	0,011	88,951
máximo	0,016	0,012	92,278	0,016	92,311
primeiro quartil	0,012	0,00825	89,842	0,012	89,878
terceiro quartil	0,014	0,00975	91,38025	0,013	91,41825
cache misses	470310 (309672 rd + 160638 wr)				
mispredict	53911420 (51907630 cond + 2003790 ind)				

4.3 Teste com 12 variáveis e 4.096 entradas

O terceiro teste realizado foi o teste do modo mega brain com 12 variáveis. O teste foi executado 10 vezes e os resultados encontrados podem ser encontrados na tabela abaixo:

Tabela 3: Medições de profiling: megabrain 12

Execução / estatística	Etapa 1 (s)	Etapa 2 (s)	Etapa 3 (s)	Etapa 4 (s)	Tempo total (s)
1	0,015	0,007	2,677	0,011	2,71
2	0,01	0,008	2,519	0,011	2,548
3	0,01	0,01	2,421	0,012	2,453
4	0,013	0,007	2,689	0,014	2,727
5	0,01	0,008	2,525	0,012	2,555
6	0,012	0,011	2,531	0,011	2,564
7	0,011	0,007	2,473	0,011	2,5
8	0,011	0,008	2,575	0,011	2,605
9	0,011	0,008	2,581	0,013	2,613
10	0,016	0,01	2,571	0,012	2,61
média	0,0119	0,0084	2,5562	0,0118	2,5885
mediana	0,011	0,008	2,551	0,0115	2,5845
Desvio padrão (populacional)	0,002022374842	0,001356465997	0,07849178301	0,0009797958971	0,08051987332
mínimo	0,01	0,007	2,421	0,011	2,453
máximo	0,016	0,011	2,689	0,014	2,727
primeiro quartil	0,01025	0,00725	2,5205	0,011	2,54975
terceiro quartil	0,01275	0,0095	2,5795	0,012	2,61225
cache misses	32676 (23657 rd + 9019 wr)				
mispredict	1827776 (1817603 cond + 10,173 ind)				

4.4 Teste com 14 variáveis e 16.384 entradas

O quarto teste realizado foi o teste do modo mega brain com 14 variáveis. O teste foi executado 10 vezes e os resultados encontrados podem ser encontrados na tabela abaixo:

Tabela 4: Medições de profiling: megabrain 14

Execução / estatística	Etapa 1 (s)	Etapa 2 (s)	Etapa 3 (s)	Etapa 4 (s)	Tempo total (s)
1	0,019	0,008	14,195	0,014	14,236
2	0,013	0,014	13,984	0,014	14,026
3	0,012	0,011	14,451	0,014	14,487
4	0,013	0,012	14,482	0,011	14,517
5	0,011	0,009	14,066	0,012	14,098
6	0,013	0,011	14,801	0,013	14,838
7	0,014	0,011	14,047	0,012	14,084
8	0,013	0,009	14,204	0,012	14,238
9	0,013	0,01	14,053	0,013	14,089
10	0,019	0,009	14,039	0,011	14,088
média	0,014	0,0104	14,2322	0,0126	14,2701
mediana	0,013	0,0105	14,1305	0,0125	14,167
Desvio padrão (populacional)	0,002607680962	0,001685229955	0,250788676	0,001113552873	0,2495000802
mínimo	0,011	0,008	13,984	0,011	14,026
máximo	0,019	0,014	14,801	0,014	14,838
primeiro quartil	0,013	0,009	14,0485	0,012	14,08825
terceiro quartil	0,01375	0,011	14,38925	0,01375	14,42475
cache misses	76282 (48785 rd + 27497 wr)				
mispredict	8291654 (8256829 cond + 34825 ind)				

5 Análise dos gargalos

Através da análise das tabelas, é possível perceber que o *hotspot* do algoritmo é definitivamente a etapa 3. Além das outras etapas executarem em um tempo consideravelmente menor, o desvio padrão também é muito maior se comparado ao da etapa 3.

A análise dos branch mispredictions constatou que a lógica de comparações não é um dos maiores problemas do algoritmo, visto que o número é pequeno se comparado ao total de predicts realizados.

Outro fator que comprova que a etapa 3 é a mais custosa computacionalmente é o gráfico de chamas, que mostra a função *percorreArvore*, chamada na etapa 3, como um grande bloco, indicando muitas chamadas e alta necessidade computacional.

6 Propostas de melhoria

A análise constatou que a etapa 3 é a parte do código que deve ser paralelizada. Como dito anteriormente, a etapa 3 consiste em um *for*, que percorre todos os casos da tabela verdade e usa a arvore gerada na etapa 2 para descobrir se o resultado informado no .txt

está válido. Os casos de teste são independentes, ou seja, um caso não necessita do resultado do outro para gerar seu próprio resultado. Sendo assim, o mesmo resultado será gerado, não importando a ordem em que os casos sejam validados.
Isso se apresenta como uma ótima característica do ponto de vista da paralelização, visto que o número total de casos pode ser separado entre os núcleos e posteriormente somados e atribuídos às variáveis contadoras de acertos e erros.

7 Conclusões

A análise dos dados elucidou algo que já era suspeitado. A etapa 3 se apresenta como sendo a maior e a que consome mais processamento entre todas as etapas. Isso já era de se esperar, visto que o numero de casos exponencialmente com o numero de variáveis. A proposta de melhoria foi paralelizar o *for* da etapa 3, visto que as iterações dele são independentes entre si.

8 Anexo 1

Página do github do projeto: Link